

3D Probabilistic Gaussian Splatting

Kacy Tran

What do you actually see?

What do you actually see?

Not pixels. Not points.

What do you actually see?

Not pixels. Not points.

Continuous surfaces

What do you actually see?

Not pixels. Not points.

Continuous surfaces

Soft edges Blended light Depth

What do you actually see?

Not pixels. Not points.

Continuous surfaces

Soft edges Blended light Depth

Reality is smooth and probabilistic

What do you actually see?

Not pixels. Not points.

Continuous surfaces

Soft edges Blended light Depth

Reality is smooth and probabilistic

Gaussian splatting models exactly this

Problem with traditional 3D reconstruction:

- Meshes / point clouds = discrete representations
- Leads to:
 - Jagged surfaces
 - Poor continuity
 - Expensive rendering

Goal:

- Continuous, differentiable scene
- Real-time rendering

(Mildenhall et al., 2020; Kerbl et al., 2023)

Key Concept

Represent a scene as a **mixture of 3D Gaussians**

$$p(\mathbf{x}) = \sum_{i=1}^N w_i \mathcal{N}(\mathbf{x} \mid \mu_i, \Sigma_i) \quad (1)$$

- $\mu_i \rightarrow$ position
- $\Sigma_i \rightarrow$ shape + orientation
- $w_i \rightarrow$ importance / opacity

(Kerbl et al., 2023)

Each point becomes a probabilistic blob

- Position: $\mu = (x, y, z)$
- Shape controlled by covariance

$$\Sigma = \begin{bmatrix} \sigma_x^2 & 0 & 0 \\ 0 & \sigma_y^2 & 0 \\ 0 & 0 & \sigma_z^2 \end{bmatrix} \quad (2)$$

- Controls spatial spread
- Enables elliptical (anisotropic) shapes

(Kerbl et al., 2023)

Key Insight

Gaussians remain Gaussians after projection

$$\Sigma' = J\Sigma J^T \quad (3)$$

- J = Jacobian of camera projection
- Result = elliptical splats on screen

(Zwicker et al., 2002; Kerbl et al., 2023)

Each Gaussian contributes to pixel color

$$C = \sum_i T_i \alpha_i c_i \quad (4)$$

- $\alpha_i \rightarrow$ opacity
- $T_i \rightarrow$ visibility
- $c_i \rightarrow$ color

Smooth blending instead of hard surfaces

(Kerbl et al., 2023)

Optimization Objective

Learning Goal

Match rendered image to real image

$$\mathcal{L} = \sum_{\text{pixels}} \|C_{\text{rendered}} - C_{\text{ground truth}}\|^2 \quad (5)$$

We optimize:

- positions μ
- shapes Σ
- colors
- opacities

(Kerbl et al., 2023)

Why It Works

Compared to point clouds

- Smooth instead of noisy
- Continuous surfaces

Compared to NeRF

- Faster
- No heavy ray marching

Real-time: 30–100 FPS with high fidelity

(Kerbl et al., 2023; Mildenhall et al., 2020)

- Dynamic scenes (time-varying Gaussians)
- Integration with AR / VR systems
- Spatial computing with Boxer
 - Real-world + digital scene alignment
 - Scene understanding + interaction layer
- Hybrid models
 - Gaussian splatting + neural fields

(Meta Research, 2024)

References

Kerbl, B., Kopanas, G., Leimkühler, T., and Drettakis, G. (2023). *3D Gaussian Splatting for Real-Time Radiance Field Rendering*. ACM Transactions on Graphics (SIGGRAPH). <https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/>

Mildenhall, B. et al. (2020). *NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis*. ECCV 2020. <https://www.matthewtancik.com/nerf>

Meta Research (2024). *Boxer: Spatial Computing Framework*. <https://facebookresearch.github.io/boxer/>

Zwicker, M. et al. (2002). *EWA Splatting*. IEEE Visualization. <https://vcg.seas.harvard.edu/publications/ewa-splatting>

